IFT339 - Pratique d'examen

Les questions suivantes évaluent si vous connaissez les structures de données vues jusqu'à présent et leur particularités, si vous pouvez déterminer quelle structure répond à vos besoins en terme de fonctionalités et de leur complexité algorithmique, et si vous parvenez à utiliser ces structures, seules ou en combinaison, pour résoudre efficacement des problèmes algorithmiques simples. Ces capacités sont observées à l'examen et démarquent ceux qui connaissent les notions de ceux qui savent les mettre en application.

Pour une préparation optimale, procédez ainsi : sur papier uniquement...

1. analysez et déterminez votre approche à l'aide de diagrammes
2. expliquez brièvement votre solution
3. écrivez le pseudo-code condensé si nécessaire
4. assurez-vous de la validité de votre solution pour tous les cas limites
5. écrivez le code C++ dans le respect des bonnes pratiques, mais sans vous soucier de la compilabilité
6. si ce n'est déjà fait, transcrivez votre réponse au propre
7. relisez votre réponse pour éviter les coquilles

Notez qu'une réponse partielle est toujours meilleure qu'une absence de réponse.

Question 1.

La nouvelle structure de données *SuperFile*, dont les éléments sont garantis être de type *comparable* (qui supportent les opérateurs de comparaison $>$, $<$ et $==$), a un comportement en tout points identique à celui de la *File* (queue) de la librairie standard.

L'on dit que *SuperFile* encapsule *File*. Elle possède en attribut une *File f*, et ses fonctions sont exactement les mêmes que la *File*. Par ailleurs, chacune délègue sa responsabilité à la fonction de la *File* qui lui est équivalente. Considérez l'exemple partiel suivant.

```
class SuperFile
    queue f
    fonction push(el)
        f.push(el)
    fonction operator[](int i)
        return f[i]
    fonction ...
        do ...
    ...
```

Vous devez déduire le reste du code C++ de la *SuperFile* et le modifier afin d'implémenter *getMin()* et *getMax()* qui retournent l'élément le plus petit et le plus grand de la *File f* en temps constant (amortis), et ce, sans nuire au fonctionnement et à la complexité en temps des autres méthodes.

Vous pouvez ajouter des attributs et modifier des méthodes de la *SuperFile*, mais vous ne pouvez pas modifier l'implémentation de la *File* standard sous-jacente. Toutes les connaissances cumulées jusqu'à présent sont accessibles. N'écrivez que l'implémentation des méthodes de la *SuperFile* pertinentes à votre solution ; ignorez celles que vous n'avez pas eu à modifier.

Temps suggéré : 5 minutes (lecture et compréhension) + 15 minutes (brouillon) + 10 minutes (transcription et vérification).

Indice

```
class SuperFile
    queue f
    deque d_min
    deque d_max
    fonction push(el)
        //À modifier
        f.push(el)
    fonction pop()
        //À modifier
        f.pop()
    fonction getMin()
        //À implémenter
    fonction getMax()
        //À implémenter
```

Fin de l'indice

Solution

Ce problème populaire est parfois utilisé pour l'évaluation des candidats durant les processus d'entrevues, ce pourquoi vous devriez vous familiariser dès que possible avec des sites d'entraînement dont *LeetCode*. La solution suivante correspond à l'approche jugée globalement la plus efficace, mais comme pour la plupart des problèmes, elle n'est pas unique. Une seconde solution utilise par exemple deux piles pour simuler l'usage de files à double extrémités de la manière observée lors d'un précédent exercice.

La solution présentée utilise deux files à double extrémités, *d_min* et *d_max*, ajoutées en attribut de la classe *SuperFile*. Nous utiliserons spécifiquement leurs capacités d'accéder à l'élément en début de file, ajouter un élément en fin de file, retirer l'élément en début de file et retirer l'élément en fin de file, le tout en temps constant.

Pour *d_min*, l'on désire maintenir tout au long de l'exécution la caractéristique suivante : l'élément en début de file doit toujours être la valeur minimale de la *File f*, la seconde doit être la deuxième plus petite valeur de la *File f*, et ainsi de suite. Du début vers la fin, les valeur de *d_min* sont donc en ordre croissant. Si l'on retire la valeur minimale de la *File f*, l'on retire simultanément la valeur en début de *d_min*, et celle qui la remplace est forcément la nouvelle valeur minimale de la *File f*. Les méthodes *getMin()* et *getMax()* ne sont donc que de simples accesseurs.

Considérant la nature arbitraire des entrées et l'impossibilité d'insertion ou de retrait en milieu de *d_min*, il devient évident que *d_min* ne peut pas conserver toutes les valeurs de la *File f*, et l'on remarque que cela n'est ni nécessaire, ni l'objectif. À l'ajout d'une valeur n dans la *File f*, l'on doit simplement retirer de *d_min* par la fin toutes les valeurs strictement supérieures à n, puis insérer également à la fin n. Ceci fonctionne puisque les valeurs retirées, préexistantes et donc prioritaires à n pour sortir de la *File f*, ne deviendront forcément jamais le minimum de par la simple présence de n dans *f*, et il est donc inutile de les conserver. Si ceci ne vous semble pas intuitif, exécutez un cas exemple sur papier jusqu'à vous en convaincre. Les manipulations concernant *d_max* sont analogues.

```
class SuperFile
    queue f
    deque d_min
    deque d_max

    fonction push(el)
        si f n'est pas vide alors
            tant que d_min n'est pas vide et que l'élément en fin de
             file est strictement plus grand que el faire
                retirer l'élément en fin de d_min
            fin
            tant que d_max n'est pas vide et que l'élément en fin de
             file est strictement plus petit que el faire
                retirer l'élément en fin de d_max
            fin
        fin
        ajouter el à la fin de d_min
        ajouter el à la fin de d_max
        ajouter el dans f

    fonction pop()
        si l'élément en début de f est équivalent à l'élément en début
         de de d_min alors
            retirer l'élément en début de d_min
        fin
        si l'élément en début de f est équivalent à l'élément en début
         de de d_max alors
            retirer l'élément en début de d_max
        fin
        retirer un élément de f

    fonction getMin()
        retourner l'élément en début de d_min

    fonction getMax()
        retourner l'élément en début de d_max
```

```cpp
using namespace std;
template <typename T>

class SuperFile {
public:
    queue<T> f;
    deque<T> d_min;
    deque<T> d_max;

    void push(T el) {
        if (f.size() > 0) {
            while (!d_min.empty() && d_min.back() > el) {
                d_min.pop_back();
            }
            while (!d_max.empty() && d_max.back() < el) {
                d_max.pop_back();
            }
        }
        d_min.push_back(el);
        d_max.push_back(el);
        f.push(el);
    }

    void pop() {
        if (f.front() == d_min.front()) {
            d_min.pop_front();
        }
        if (f.front() == d_max.front()) {
            d_max.pop_front();
        }
        f.pop();
    }

    T getMin() { return d_min.front(); }
    T getMax() { return d_max.front(); }
};
```

Les liens suivants fournissent la même solution, mais avec des explications différentes. Vous noterez que le code n'est pas aussi clair et concis, deux caractéristiques auxquelles vous devriez porter attention.

`https://www.geeksforgeeks.org/design-a-queue-data-structure-to-get-minimum-or-maximum-in-o1-time/`

`https://stackoverflow.com/questions/12054415/get-min-max-in-o1-time-from-a-queue`

La solution à ce problème est simple, et il est dangereux d'assumer que puisqu'elle est facile à comprendre, elle est aussi facile à conçevoir. C'est ce pourquoi la pratique est aussi, sinon plus, importante que la lecture.

Fin de la solution

Question 2.

Chaque noeud d'une liste chaînée est étiquetté *Alpha* ou *Beta*. Une telle liste est dite *bien formée* si, dans un parcours itéré, tous ses noeuds étiquetés *Alpha* se situent avant ceux étiquetés *Beta*.

Écrivez le code C++ d'une fonction qui reçoit en paramètre un pointeur Noeud* *lsc* représentant le début d'une liste chaînée dont tous les noeuds sont étiquetés *Alpha* ou *Beta* dans un ordre arbitraire, et retourne un pointeur Noeud* vers la liste chaînée réordonnée de telle sorte qu'elle soit maintenant *bien formée*.

Vous devez réaliser cette tâche en place. Soit, vous ne pouvez pas créer de nouveau noeuds en mémoire. Vous ne pouvez que manipuler les adresses auxquelles ceux de la chaîne font référence. Déclarer de nouvelles variables de type primitif ou pointeur est autorisé et requis.

Pour tout noeud *node*, les accesseurs *node->is_alpha()* et *node->is_beta()* vous sont disponibles. Les étiquettes des noeuds ne devraient pas être comparées autrement.

Vous aurez tous vos points pour une solution valide dont la complexité en temps est linéaire en n, la quantité de noeuds dans la liste chaînée passée en paramètre, et si les cas limites sont gérés.

Temps suggéré : 5 minutes (lecture et compréhension) + 15 minutes (brouillon) + 10 minutes (transcription et vérification).

Indice

```
fonction reorder(Noeud* lsc)
    Noeud* current = lsc
    Noeud* first_alpha, last_alpha = nullptr
    Noeud* first_beta, last_beta = nullptr
    tant que current est un pointeur valide faire
        si current est étiqueté Alpha alors
            //À modifier
        fin
        sinon
            //À modifier
        fin
        current = current->next
    fin
    si first_alpha n'est pas un pointeur valide alors
        //À modifier
    fin
    sinon si first_beta est un pointeur valide alors
        //À modifier
    fin
    retourner first_alpha
```

Fin de l'indice

Solution

Cette solution demande un brin d'ingéniosité. L'approche suggérée pour sa complexité linéaire est de localiser les deux premiers noeuds étiquetés *Alpha* et *Beta*, et maintenir deux nouvelles sous-listes chaînées débutant avec ces noeuds, lesquelles seront ensuite tout simplement jointes. Notez que ces nouvelles listes ne sont pas nouvelles en mémoire. Elle sont uniquement composées des noeuds de la liste initiale, et ce, sans même les déplacer en mémoire. Seules les adresses référencées sont modifiées.

Décrit plus simplement, un itérateur parcours la liste chaînée *lsc*. L'on conserve un pointeur vers le *premier* noeud *Alpha*. L'on dénote ce noeud *dernier* pour le moment. Au second noeud *Alpha*, l'on modifie les adresses de ce noeud et de celui anoté *dernier* pour qu'ils se suivent. Ce second noeud devient le nouveau *dernier*. L'on continue ainsi parallèlement avec les noeuds étiquetés *Beta* jusqu'à terminer l'itération avec quatre pointeurs vers le début et la fin des deux listes, et qu'il soit ainsi possible de les joindre.

N'oubliez surtout pas de gérer les cas limites. Dans ce scénario, les deux cas limites possibles sont que la liste chaînée pouvait ne contenir que des noeuds dont les étiquettes sont toutes les mêmes, soit *Alpha* ou *Beta*. Ainsi, il serait dangereux de tenter de joindre deux sous-listes à la fin dont l'une n'existe pas.

De nouveau, assurez-vous de bien comprendre l'intuition et le code de cette solution.

```
fonction reorder(Noeud* lsc)
    Noeud* current = lsc
    Noeud* first_alpha, last_alpha = nullptr
    Noeud* first_beta, last_beta = nullptr
    tant que current est un pointeur valide faire
        si current est étiqueté Alpha alors
            si first_alpha n'est pas un pointeur valide alors
                first_alpha = current
            fin
            sinon
                last_alpha->next = current
            fin
            last_alpha = current
        fin
        sinon
            si first_beta n'est pas un pointeur valide alors
                first_beta = current
            fin
            sinon
                last_beta->next = current
            fin
            last_beta = current
        fin
        current = current->next
    fin
    si first_alpha n'est pas un pointeur valide alors
        retourner first_beta
    fin
    sinon si first_beta est un pointeur valide alors
        last_alpha->next = first_beta
        last_beta->next = nullptr
    fin
    retourner first_alpha
```

```
using namespace std;

Noeud* reorder (Noeud* lsc) {
    Noeud* current = lsc;
    Noeud* first_alpha = nullptr, last_alpha = nullptr;
    Noeud* first_beta = nullptr, last_beta = nullptr;
    while (current) {
        if (current->is_alpha) {
            if (!first_alpha)
                first_alpha = current;
            else
                last_alpha->next = current;
            last_alpha = current;
        }
        else {
            if (!first_beta)
                first_beta = current;
            else
                last_beta->next = current;
            last_beta = current;
        }
        current = current->next;
    }
    if (!first_alpha)
        return first_beta;
    else if (first_beta) {
        last_alpha->next = first_beta;
        last_beta->next = nullptr;
    }
    return first_alpha;
}
```

Fin de la solution

Question 3.

Déterminez la complexité algorithmique en temps des fonctions suivantes.

```
using namespace std;

void algo_1 (size_t k, n, m) {
    for (size_t i = 0; i < n; i++) {
        for (size_t j = 0; j < m; j++) {
            if (k > 0) {
                algo_1(k-1, n, m);
            }
        }
    }
}

void algo_2 (size_t a, b) {
    while (a < b) {
        for (size_t i = 999999; i > 0; i--) {
            a++;
            b--;
        }
    }
}
```

Temps suggéré : 5 minutes (lecture et compréhension) + 10 minutes (écriture et vérification).

Solution

La complexité de la première fonction est $O(n^k m^k)$. Une manière de développer l'intuition est de dessiner un arbre des appels récursifs. Sa profondeur sera d'au maximum k suivant la condition interne. À chaque pallier, les $n * m$ appels à la fonction provoquent $n * m$ nouveaux appels, et ce, jusqu'à l'atteinte de la condition de fin. Nous multiplions donc $n * m$ par $n * m$ un total de k fois, ce qui se simplifie pour donner la complexité attendue.

La complexité de la seconde fonction est $O(n)$, où n est la distance normalisée entre a et b. Bien que a et b se rapprochent à une très grande vitesse constante C, cela n'importe pas puisque ces éléments peuvent aussi être infiniment éloignés, ce pourquoi il faut toujours éviter de tenter d'analyser la complexité avec des valeurs concrètes. Mathématiquement, si l'on calcule la véritable valeur de n, soit $|b - a|$ (ou $|a - b|$) divisé par deux fois la constante C, l'on réalise que le diviseur reste constant et qu'il doit être ignoré. L'on termine avec $|b - a|$ (ou $|a - b|$), valeur composée de deux variables, et donc variable en soi, que l'on peut associer à n dans notre analyse. Rappellez-vous que ce que vous représentez avec n ou m est de votre choix, mais vous devez rester conséquent.

Fin de la solution

Conseils pour tout examen universitaire

S'il vous est autorisé d'emmener toute documentation papier, imprimez tout ce que vous pouvez, incluant les solutions aux exercices, les devoirs et les examens des années précédentes. Ceci peut parfois représenter plus d'une centaine de pages. Cela ne sera pas toujours utile, mais vous pourriez être ainsi en mesure de répondre à une question qui vous donnera avantage sur vos collègues.

Plusieurs proscriront d'emmener trop de documentation puisqu'il est facile de s'y perdre, et que vous devriez en connaître le contenu par coeur si vous êtes réellement prêts à l'examen. Ils ont en partie raison. Qu'importe la quantité de documentation que vous choisissez d'emmener, elle devrait être très méticuleusement classée au préalable. Vous ne pouvez pas en mémoriser l'ensemble du contenu, surtout alors que vous avez plusieurs examens, mais vous devez absolument savoir exactement où trouver une information spécifique instantanément.

Que cela soit demandé ou non, et que vous emmeniez toute autre documentation ou non, écrivez toujours une feuille de notes résumées recto-verso. Une bonne feuille de notes contiendra tout ce dont vous avez de besoin et vous n'aurez jamais à vous référer à votre documentation. Une meilleure feuille de notes ne sera jamais consultée à l'examen.

Bien qu'il soit fortement recommandé de pratiquer avec chronomètre les examens des années précédentes lorsqu'ils vous sont accessibles, prenez au moins le temps de les lire et de les comprendre. Les questions sont parfois similaires, et vous donneront une bonne idée de ce à quoi vous attendre le jour de l'examen en termes de difficulté, contenu évalué, répartition et durée globale. Parvenir à compléter un examen de pratique vous garantis la réussite du véritable examen, ne serait-ce que par la sérénité que cela vous apportera.

Lors de l'examen, ne perdez pas une seule seconde. Parcourir l'ensemble des questions une première fois est un réflexe inutile. Aucun examen n'est piégé, et vous devriez être en mesure de les compléter dans l'ordre dans les temps sauf si votre préparation est insuffisante. Les questions sont généralement classées de la plus facile à la plus difficile, de la plus courte à la plus longue, ou de sorte à faciliter l'évaluation des capacités et la correction.

La longueur de l'examen dépend du style de l'enseignant ; certains préfèrent vérifier l'habileté à réfléchir et parvenir à une solution logique déduite durant l'examen, alors que d'autres vérifient plutôt la mémorisation et la vitesse d'exécution. Dans ce second cas, il est même possible, et attendu, qu'aucun étudiant ne termine l'examen. Ne paniquez pas si cela survient, mais retenez que l'avantage est à celui ou celle qui se rend le plus loin.

Les réponses que vous donnerez à l'examen devraient respecter la forme des six C : claires, complètes, concises, complètes, concises et claires. Oui, chaque élément est important au point d'être listé deux fois.

En débutant par la première question et en poursuivant dans l'ordre ;

1. Si vous connaissez la réponse, écrivez-la maintenant et passez à la question suivante.
2. Si vous savez exactement où trouver la réponse, écrivez ou transcrivez-la maintenant et passez à la question suivante.
3. Si ni 1 ni 2 n'est applicable, passez à la question suivante.
4. À la fin, revenez sur les questions qui vous semble possibles, et espérez survivre à votre examen.

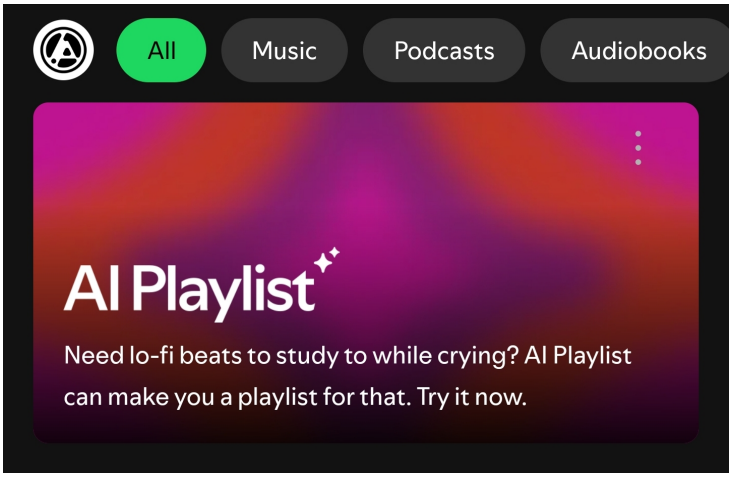
En somme, retenez ceci ; si vous devez réfléchir un certain temps, c'est probablement normal. Si vous devez chercher, ça ne l'est pas.

Plusieurs examens ont un espace brouillon non-évalué à l'endos des pages, à la gauche de la question. Emmenez toujours des feuilles brouillon au cas où ce ne soit pas le cas. Allouez un temps limité à l'écriture d'un brouillon en gardant en tête qu'en cas de réussite, il devra être transcrit. Dessinez les structures, testez un pseudo-code, exécutez un code donné... les utilités du brouillon sont multiples.

Assurez-vous d'avoir répondu à toutes les questions. Il peut survenir qu'une dernière série de questions se trouve en dernière page, au verso. Ce n'est de nouveau pas un piège. Si des questions rapides en début d'examen servent à vérifier les connaissances de base nécessaires à une note de passage, des questions rapides à la fin servent plutôt à départager les meilleurs étudiants du reste du lot qui pourrait manquer de temps pour y répondre.

Par ailleurs, gérez votre temps. Vous devez considérer que chaque question doit être lue, relue, comprise, explorée, solutionnée, trancrite et vérifiée. Si vous accordez plus de temps qu'il n'en faut pour une étape, vou n'aurez pas le temps nécessaire pour compléter l'ensemble.
Finalement, révisez l'ensemble de vos réponses, juste au cas. Il n'y a que deux excuses pour quiter un examen avant la fin du temps alloué ; vous avez prédit avoir $A+$ en lisant les feuilles de thé le matin même, ou vous avez moins que $A+$, et vous êtes prêts à vivre avec le fait que vous pouviez faire mieux, et vous avez choisi le contraire.

Bons examens, et bonne réussite (à qui travaillera pour!).



*Capture d'écran véritable. Merci Spotify!